جامعة الإمام عبدالرحمن بن فيصل
IMAM ABDULRAHMAN BIN FAISAL UNIVERSITY

College of Science and humanities

Computer Department

Term 2, 2024-2025

CS422 Language Theory & Finite Automata

PacMan Automaton

Student Name	ID Number

Supervised by:

Dr. Azzah Ahmed Ali

Contents

Project Phase 1 5

1. real-life application of Language Theory & Finite Automata 5

1.1. Description of the system including definition of each function of the system ... 5

1.2. Description of the language represented the system 5

1.3. One example of a string that belongs to the language 8

1.4. One example of a string that does not belong to the language. 8

2. Simplified version of this application: 9

2.1. NFA. 9

2.2. Definition of NFA-(Mathematical model). 12

2.3. One example of a string that accepted from NFA. 12

2.4. One example of a string that does not that accepted from NFA. 13

Phase 2 : DFA Automata and Grammar Development 14

1. Designing Deterministic Finite Automata (DFA) for Language Strings : 14

1.1. **DFA graph and Transition Table:** 14

1.1.4. String Acceptance Analysis 17

1.1.5. Accepted Strings DFA: 17

1.1.6. Rejected Strings DFA: 17

2. Formulating Production Rules for Language Grammar : 18

2.1. Grammar rules of NFA : 18

2.2. Grammar rules of DFA : 20

Phase 3 : Syntax Analyzer Of Language 21

1. Design The grammar code: In this phase, we used ANTLR inside IntelliJ to convert grammar rules into Java code. This code checks if a test string matches the grammar and generates a parse tree when accepted. The tree shows how the input string follows the rules step by step. 21

1.1. Coding : 21

1.1.1. DFA code : 21

1.1.1.1. Accept strings : 21

1.1.1.2. Rejected strings : 22

.1.1.2 NFA code : 23

1.1.2.1. Accept strings : 23

1.1.2.2. Rejected strings : 23

1.2. Website : 24

References : 26

Table of Tabels

Table 1 Functions of Transition (δ) Table 8
Table 2 Alphabet Explanation Table 9
Table 3 States description 10
Table 4 Full NFA Transition Table 11
Table 5 DFA Transition Table 14
Table 6 Functions of Transition (δ) Table in DFA 15
Table 7 DFA Alphabet Description 15
Table 8 DFA State Description 16

Table of figures

Figure 1 NFA Representation of Pac-Man and Ghosts 12
Figure 2 example of a string that accepted from NFA. 12
Figure 3 example of a string that does not that accepted from NFA 13
Figure 4 DFA Representation of Pac-Man and Ghosts 15
Figure 5 Accepted Strings DFA 17
Figure 6 Accepted Strings DFA 17
Figure 7 DFA Accept strings 21
Figure 8 DFA Accept strings (Win state) 22
Figure 9 DFA Rejected strings Example 22
Figure 10 NFA Accept strings Example 23
Figure 11 NFA Rejected strings Example 24
Figure 12 grammer mode 24
Figure 13 perse tree for grammar 25
Figure 14 input string perse tree 25

Project Phase 1

1. real-life application of Language Theory & Finite Automata

1.1. Description of the system including definition of each function of the system

The system is designed to simulate Pac-Man's movements, interactions and activities reflecting the mechanics of the Pac-Man video game. Using non-deterministic finite automata (NFA), the system models Pac-Man's behavior as he navigates the maze, encounters ghosts, collects points, and interacts with environmental changes.
Pac-Man starts in a starting state (S) and moves through Movement (M), Immunity (I), Winning (W), and various ghost interactions. The ghosts also follow a structured behavior, switching between waiting (G_W), chasing (G_C), escaping (G_F), being eaten (G-E), and returning (G_R). The game ends when Pac-Man loses all lives (GameOver) or wins by consuming all the points (W).

1.2. Description of the language represented the system

The system is made to mimic Pac-Man's movements, activities, and interactions in a grid-based setting, much like the Pac-Man video game. The language is broken down here, along with its alphabet and how it relates to the system's operations:

Alphabet (Σ)

The game's potential actions and environmental events are represented by the alphabet:

•Σ = {U, D, L, R, E, G, P, F, T, AllDotsEaten, Restart, GhostStart, SeePac, CaughtByPac, ReachedBase}

- U: Move up
- D: Move down
- L: Move left
- R: Move right
- E: Eat dot (Pac-Man consumes a regular dot)
- G: Encounter ghost (potentially losing a life)
- P: Consume Power Pellet (makes Pac-Man temporarily invincible)
- F: Consume fruit (bonus points)
- T: Timer expires (used for power pellet effect duration)
- AllDotsEaten: Player has consumed all dots, winning the game
- Restart: Player starts a new game
- GhostStart: A ghost spawns and starts moving
- SeePac: A ghost sees Pac-Man and starts chasing
- CaughtByPac: An invincible Pac-Man eats a ghost
- ReachedBase: A ghost returns to its base after being eaten

2. States (Q)

Pac-Man's situations within the system are represented by the automaton's states:
Q = {S, M, I, D1, D2, D3, GameOver, W, G_W, G_C, G_F, G_E, G_R}

- S: Start state
- M: Moving state (Pac-Man moves normally)
- I: Invincible state (Pac-Man has eaten a power pellet)
- D1, D2, D3: Death states (represent remaining lives)
- GameOver: Terminal state (Pac-Man has lost all lives)
- W: Winning state (Pac-Man eats all dots)
- G_W: Ghost waiting state
- G_C: Ghost chasing Pac-Man
- G_F: Ghost fleeing after Pac-Man eats a power pellet
- G_E: Ghost has been eaten by an invincible Pac-Man
- G_R: Ghost returns to its base

Initial State (q_0)

- q_0 = S (Start state)

Accepting States (F)

- F = {M, I, W}
- M: Pac-Man is actively playing.
- I: Pac-Man is invincible.
- W: Pac-Man has won the game.
- G_C: Ghost is chasing Pac-Man.
- G_F: Ghost is fleeing after Pac-Man eats a power pellet.

3. Functions of Transition (δ)

The role of transition $\delta: Q \times \Sigma \rightarrow Q$ controls the system's transitions between states in response to Pac-Man's actions and the surroundings:

Description	Next State	Input (Σ)	Current State
Pac-Man starts moving	M	U,D,L,R	S
Normal movement	M	U,D,L,R	M
Pac-Man eats a dot and continues	M	E	M
Pac-Man eats a power pellet, becoming invincible	I	P	M
Pac-Man collides with a ghost, losing a life	D1,D2,D3	G	M
Power pellet effect ends, returning Pac-Man to normal	M	T	I
Pac-Man eats a ghost while invincible	M	G	I
Restart the game after losing a life	S	RESTART	D1
Restart the game after losing all lives	S	RESTART	D3
Pac-Man loses all lives, game over	GameOver.	G	D3

Pac-Man eats all dots and wins	W	AllDotsEaten	M/I
Ghost sees Pac-Man and starts chasing	G_C	SeePac	G_W
Pac-Man eats a power pellet, ghost starts fleeing	G_F	P	G_C
Pac-Man eats the ghost	G_E	GaughtByPac	G_F
Ghost respawns at base	G_W	ReachedBase	G_E

Table 1 Functions of Transition (δ) Table

4. Description of Language (L)

All legitimate action sequences that Pac-Man can do within the system are included in the language L that this automaton represents. Combinations of moves, eating dots, drinking power pellets, and running into ghosts could all be considered viable sequences as long as state transitions adhere to the game's regulations.

1.3.One example of a string that belongs to the language

"U E L E D R F" → Pac-Man moves up, eats a dot, moves left, eats another dot, moves down, moves right, and eats a fruit.

1.4.One example of a string that does not belong to the language.

Example of an invalid string: "P G" (beginning with a power pellet consumption that is motionless).

2. Simplified version of this application:

2.1.NFA.

In this section, we present a **Non-Deterministic Finite Automaton (NFA)** that models the behavior of **Pac-Man and the ghosts** within the game environment. The NFA captures all possible state transitions based on **Pac-Man's movements, interactions with objects (dots, power pellets, fruits), and encounters with ghosts**.as it shown in Table 1 and 2 .

Alphabet	Description
U	(Move Up)
D	(Move Down)
L	(Move Left)
R	(Move Right)
E	(Eat Dot)
G	(Meet a Ghost)
P	(Eat Power Pellet)
F	(Eat Fruit)
T	(Power Pellet Timer Ends)
AllDotsEaten	(All Dots Eaten)
Restart	(Restart the Game)
GhostStart	(Ghosts Start Moving)
SeePac	(Ghosts See Pac-Man)
CaughtByPac	(Pac-Man Catches a Ghost)
ReachedBase	(Ghost Returns to Base)

Table 2 Alphabet Explanation Table

State	Description
S	(Start State)
M	(Moving Normally)
I	(Invincible After Power Pellet)
D1	(First Death State)
D2	(Second Death State)
D3	(Third Death State)
GameOver	(Game Over State)
W	(Winning State)
G_W	(Ghost Waiting State)
G_C	Pac-Man (Ghosts Chasing Pac-Man)
G_F	(Ghosts Running Away)
G_E	Pac-Man (Ghosts Eaten)
G_R	(Ghosts Returning to Base)

Table 3 States description

This **NFA serves as a foundation** for modeling the Pac-Man game in Figure 1 and Table 3 Full NFA Transition Table .

State	U	D	L	R	E	G	P	F	T	AllDots Eaten	Restart	Ghost Start	See Pac	Caught ByPac	ReachedBase
S	M	M	M	M	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø
M	M	M	M	M	M	D1	G_F,I	M	Ø	W	Ø	Ø	Ø	Ø	Ø
G_F,I	I	I	I	I	Ø	M	G_F,I	-	G_C, M	Ø	Ø	Ø	Ø	G_E	Ø
G_E	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	G_R
G_R	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	G_C	Ø	Ø
G_C	Ø	Ø	Ø	Ø	Ø	Ø	G_F	Ø	Ø	Ø	Ø	Ø	G_C	Ø	Ø
G_F	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	G_C	Ø	Ø	Ø	Ø	G_E	Ø
G_C, M	M	M	M	M	M	D1	G_F,I	M	Ø	W	Ø	Ø	G_C	Ø	Ø
I	I	I	I	I	Ø	M	G_F,I	Ø	G_C, M	Ø	Ø	Ø	Ø	Ø	Ø
D1	Ø	Ø	Ø	Ø	Ø	D2	Ø	Ø	Ø	Ø	G_C, M	Ø	Ø	Ø	Ø
D2	Ø	Ø	Ø	Ø	Ø	D3	Ø	Ø	Ø	Ø	G_C, M	Ø	Ø	Ø	Ø
D3	Ø	Ø	Ø	Ø	Ø	Game Over	Ø	Ø	Ø	Ø	G_C, M	Ø	Ø	Ø	Ø
Game Over	Game Over	Game Over	Game Over	Game Over	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø	Ø

Table 4 Full NFA Transition Table

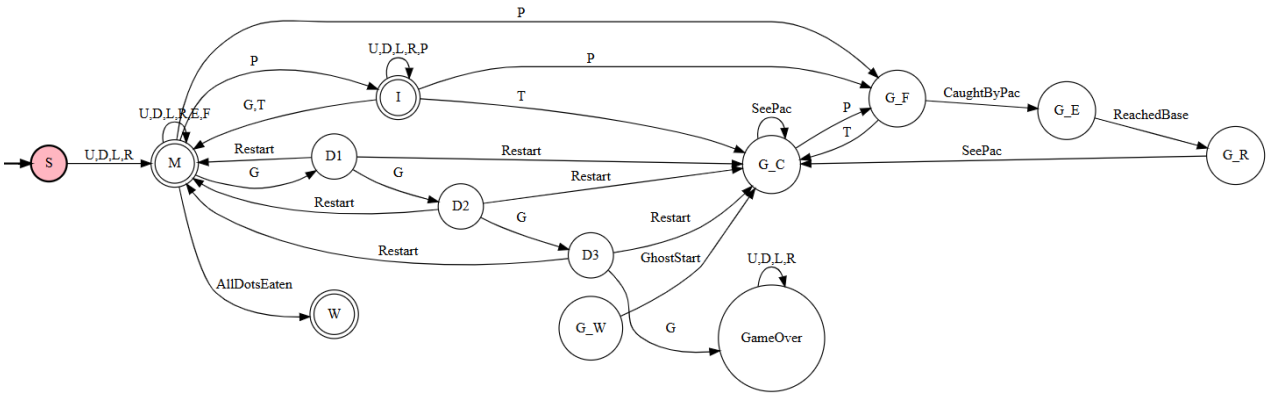


Figure 1 NFA Representation of Pac-Man and Ghosts

2.2. Definition of NFA-(Mathematical model).

Q → Finite set of states → {S, M ,1, W, D1, D2, D3, G_F, G_C, G_W, G_E, G_R, GameOver}

Σ → Input alphabet → {U,D,L,R,P,G,T}

q_0→ Starting / Initial state → {S}

F → set of final state → {1, M, W,}

2.3. One example of a string that accepted from NFA.

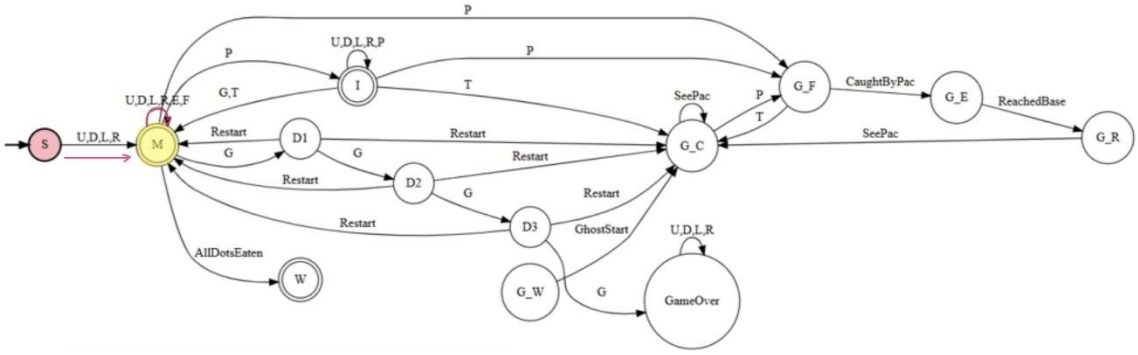


Figure 2 example of a string that accepted from NFA.

the sequence on the diagram using the following explanation:

We start from state S (the pink node).

We move to M because the arrow from S to M supports the movements U, D, L, R, E, F.

We remain in M while executing the successive moves, as all the actions (U, E, L, E, D, R, F) are supported within state M.

Finally, the sequence ends in state M, which is an accepted state.

2.4.One example of a string that does not that accepted from NFA.

String:

P GP

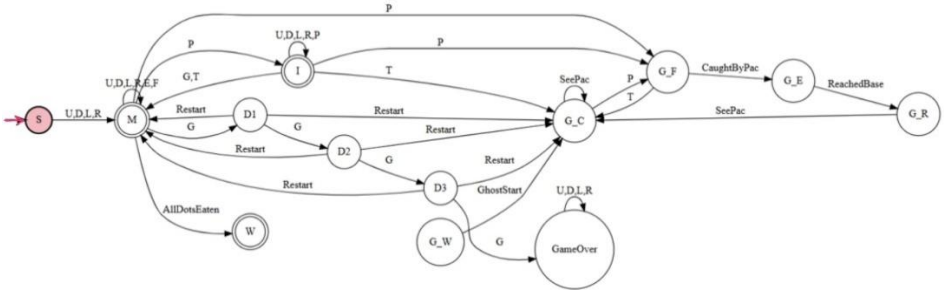


Figure 3 example of a string that does not that accepted from NFA

Sequence of Steps on the Diagram:

1. **P (Power Pellet)**

- We start from state **S** (the initial state).
- According to the diagram, there is no direct transition from **S** to any other state using P because Pac-Man must move first (U, D, L, R).

- Since `P` is not allowed as the first action without movement, Pac-Man cannot transition to any valid state.
- He will not reach any accepted final state (such as winning or continuing the game).
- The diagram does not contain a transition that allows "P" directly from `S`, proving that this string **is not accepted.**

Phase 2 : DFA Automata and Grammar Development

1. Designing Deterministic Finite Automata (DFA) for Language Strings :

In this section, we previously introduced a NFA in Phase 1, as depicted in **Figure 1** and detailed in **Table 4**. We will now proceed to convert this NFA into a DFA that models the behavior of Pac-Man and the ghosts within the game environment.

1.1. DFA graph and Transition Table:

State	U	D	L	R	E	G	P	F	T	AllDots Eaten	Rest art	Ghost Start	See Pac	Caught ByPac	Reache dBase
S	M	M	M	M	De ad	Dead	De ad	De ad	Dea d	Dead	Dea d	Dead	Dea d	Dead	Dead
M	M	M	M	M	M	D1	G_ F,I	M	Dea d	W	Dea d	Dead	Dea d	Dead	Dead
I	I	I	I	I	De ad	M	G_ F,I	De ad	G_ C,M	Dead	Dea d	Dead	Dea d	Dead	Dead
D1	Dead	Dead	Dead	Dead	De ad	D2	De ad	De ad	Dea d	Dead	G_C ,M	Dead	Dea d	Dead	Dead
D2	Dead	Dead	Dead	Dead	De ad	D3	De ad	De ad	Dea d	Dead	G_C ,M	Dead	Dea d	Dead	Dead
D3	Dead	Dead	Dead	Dead	De ad	Game Over	De ad	De ad	Dea d	Dead	G_C ,M	Dead	Dea d	Dead	Dead
Game Over	Game Over	Game Over	Game Over	Game Over	De ad	Dead	De ad	De ad	Dea d	Dead	Dea d	Dead	Dea d	Dead	Dead
W	Dead	Dead	Dead	Dead	De ad	Dead	De ad	De ad	Dea d	Dead	Dea d	Dead	Dea d	Dead	Dead
G_W	Dead	Dead	Dead	Dead	De ad	Dead	De ad	De ad	Dea d	Dead	Dea d	G_C	Dea d	Dead	Dead
G_C	Dead	Dead	Dead	Dead	De ad	Dead	G_ F	De ad	Dea d	Dead	Dea d	Dead	G_ C	Dead	Dead
G_F	Dead	Dead	Dead	Dead	De ad	Dead	De ad	De ad	G_ C	Dead	Dea d	Dead	Dea d	G_E	Dead
G_E	Dead	Dead	Dead	Dead	De ad	Dead	De ad	De ad	Dea d	Dead	Dea d	Dead	Dea d	Dead	G_R
G_R	Dead	Dead	Dead	Dead	De ad	Dead	De ad	De ad	Dea d	Dead	Dea d	Dead	G_ C	Dead	Dead
G_C, M	M	M	M	M	M	D1	G_ F,I	M	Dea d	W	Dea d	Dead	G_ C	Dead	Dead
G_F,I	I	I	I	I	De ad	M	G_ F,I	De ad	G_ C,M	Dead	Dea d	Dead	Dea d	G_E	Dead
Dead	Dead	Dead	Dead	Dead	De ad	Dead	De ad	De ad	Dea d	Dead	Dea d	Dead	Dea d	Dead	Dead

Table 5 DFA Transition Table

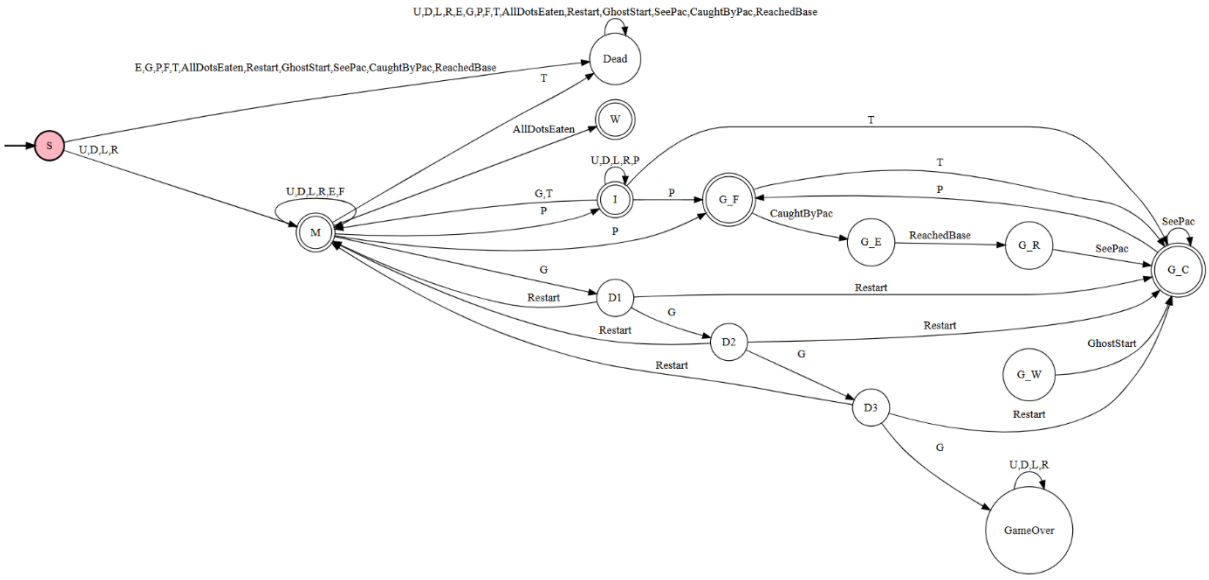


Figure 4 DFA Representation of Pac-Man and Ghosts

1.1.1. Functions of Transition (δ) Table

Current State	Input (Σ)	Next State	Description
S	U, D, L, R	M	Pac-Man moves in any direction
M	P	G_F,I	Pac-Man eats power pellet, ghosts become frightened
M	G	D1	Pac-Man encounters a ghost
M	AllDotsEaten	W	All dots collected, win state
I	T	G_C,M	Frightened ghosts return to chase mode
D3	G	GameOver	Pac-Man gets caught, game over
G_E	ReachedBase	G_R	Ghost returns to base after being eaten
G_R	SeePac	G_C	Ghost resumes chase mode

Table 6 Functions of Transition (δ) Table in DFA

1.1.2. Alphabet Description

Symbol	Description
U, D, L, R	Pac-Man moves Up, Down, Left, Right
E	Eat pellet
G	Encounter ghost
P	Power pellet
F	Frightened ghost mode
T	Timer for frightened mode ends
AllDotsEaten	All dots collected
Restart	Restart game
GhostStart	Ghosts start moving
SeePac	Ghost sees Pac-Man
CaughtByPac	Ghost caught by Pac-Man
ReachedBase	Ghost returns to base

Table 7 DFA Alphabet Description

1.1.3. State Description

State	Description
S	Start state
M	Pac-Man moving normally
I	Pac-Man invincible (power pellet active)
D1, D2, D3	Ghost chasing states
GameOver	Pac-Man loses the game
W	Win state — all dots eaten
G_W	Ghost waiting to start
G_C	Ghost chasing Pac-Man
G_F	Ghost frightened by power pellet
G_E	Ghost eaten by Pac-Man
G_R	Ghost returning to base
G_C,M	Mixed state — ghost chasing and Pac-Man moving
G_F,I	Mixed state — ghost frightened, Pac-Man invincible
Dead	Invalid transitions (trap state)

Table 8 DFA State Description

1.1.4. String Acceptance Analysis

1.1.5. Accepted Strings DFA:

Accepted string : DPUURUUUPUPLLPD

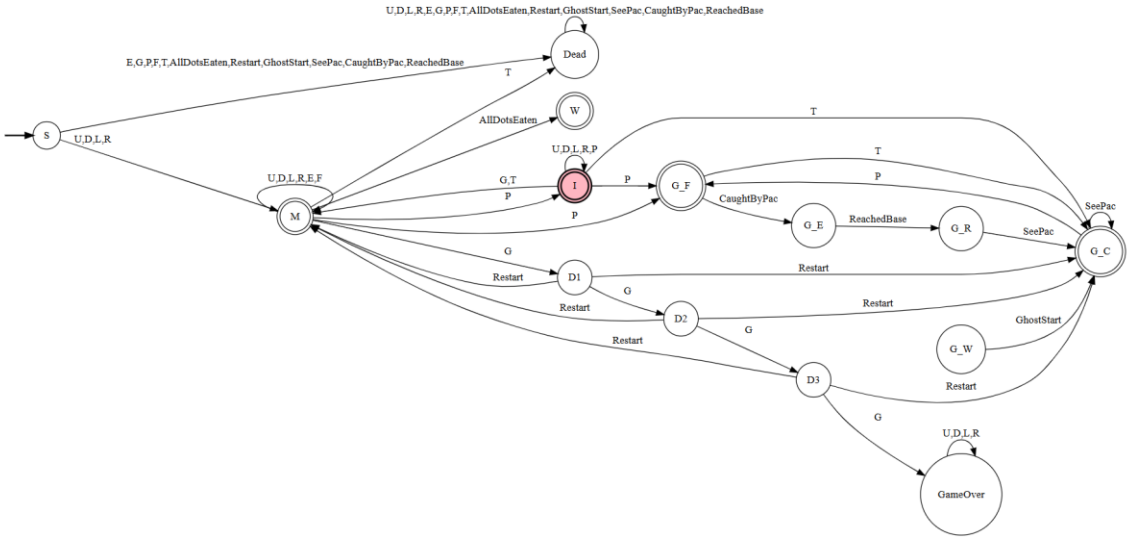


Figure 5 Accepted Strings DFA

1.1.6. Rejected Strings DFA:

Accepted string : URGG

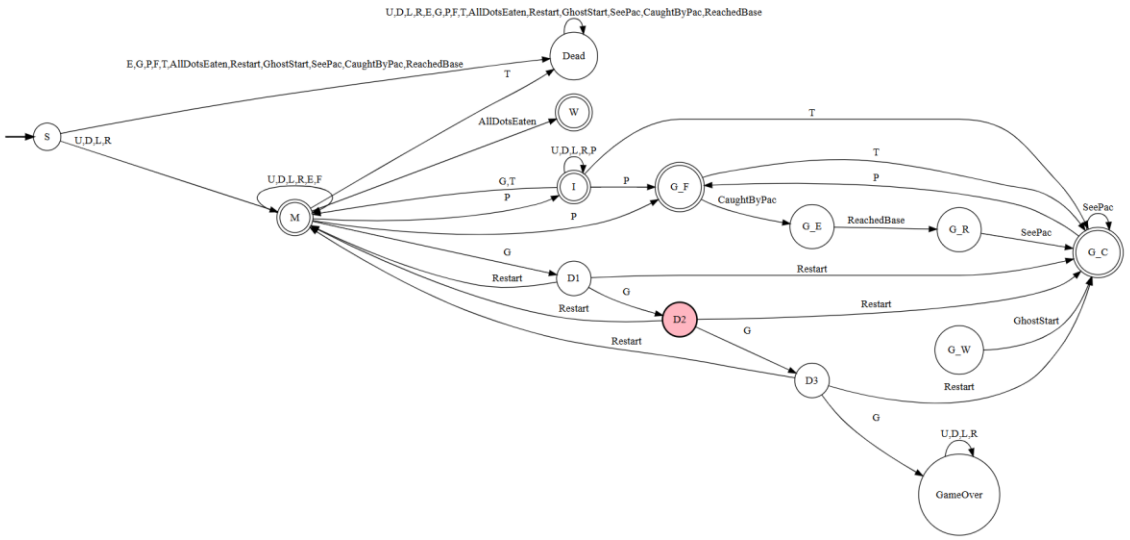


Figure 6 Accepted Strings DFA

2. Formulating Production Rules for Language Grammar :

2.1. Grammar rules of NFA :

A grammar G = (V, Σ, R, S) consists of:

V: Set of variables (non-terminals)

Σ: Set of terminals (alphabet of the NFA)

R: Set of production rules

S: Start symbol

Σ = {U, D, L, R, E, F, P, T, G, G_E, G_R, G_W, G_C, G_F, SeePac, AllDotsEaten, Restart}

V = {S, Game, M, Invincible, PowerInteraction, GhostBehavior, GhostChase, GhostFlee, GhostEaten, Death1, Death2, Death3, Win, Restart, GameOver}

S: Start symbol of the grammar

Game: Represents the overall game flow including movement, deaths, ghost behavior, winning, and restarting

M: A sequence of Pac-Man's moves and interactions in the maze

Invincible: State where Pac-Man becomes invincible after eating a power pellet

PowerInteraction: Interactions during the invincible state (e.g., eating ghosts, timer ending)

GhostBehavior: Describes how ghosts react to Pac-Man's state (wait, chase, flee, etc.)

GhostChase: Ghost chasing Pac-Man after detecting him

GhostFlee: Ghost fleeing from invincible Pac-Man

GhostEaten: Sequence after ghost is eaten by Pac-Man and returns to base

Death1: First death state for Pac-Man (D1)

Death2: Second death state for Pac-Man (D2)

Death3: Third death state for Pac-Man (D3)

Win: Represents the winning condition (all dots eaten)

Restart: Restarting the game after losing a life

GameOver: End of the game after all lives are lost

Start symbol = S

Rules:

S → Game

Game → M Game

| Invincible Game

| Death1 Game

| Death2 Game

| Death3 GameOver

| GhostBehavior Game

| Win

| ε

M → U M | D M | L M | R M | E M

| F M

| P Invincible

| ε

Invincible → I PowerInteraction

PowerInteraction → G_E G_R | T

GhostBehavior → G_W GhostChase

GhostChase → SeePac G_C | P GhostFlee

GhostFlee → G_F G_E G_R G_W

Death1 → D1 Restart Game

Death2 → D2 Restart Game

Death3 → D3

Win → AllDotsEaten W

GameOver → GameOver

Restart → Restart

2.2.Grammar rules of DFA :

A grammar G = (V, Σ, R, S) consists of:

V: Set of variables (non-terminals)

Σ: Set of terminals (alphabet of the NFA)

R: Set of production rules

S: Start symbol

V = {S, M, Dead, W, I, G_F, G_E, G_R, G_C, D1, D2, D3, G_W, GameOver}

Σ = {U, D, L, R, E, G, P, F, T, AllDotsEaten, Restart, GhostStart, SeePac, CaughtByPac, ReachedBase}

Start symbol = S

S → U M | D M | L M | R M

M → U M | D M | L M | R M | E M | G M | P M | F M | T Dead | AllDotsEaten W | Restart D1 | GhostStart G_W | SeePac G_C | CaughtByPac G_E | ReachedBase G_R | ε

W → ε

Dead → ε

I → G G_F | T G_C | P M

G_F → CaughtByPac G_E

G_E → ReachedBase G_R | Restart G_C

G_R → SeePac G_C | Restart G_C

G_C → SeePac G_C | Restart G_W | GhostStart G_W | ε

G_W → Restart G_C

D1 → Restart D2

D2 → Restart D3

D3 → G GameOver

GameOver → U GameOver | D GameOver | L GameOver | R GameOver | ε

Phase 3 : Syntax Analyzer Of Language

1. Design The grammar code:

In this phase, we used ANTLR inside IntelliJ to convert grammar rules into Java code. This code checks if a test string matches the grammar and generates a parse tree when accepted. The tree shows how the input string follows the rules step by step.

1.1.Coding :

1.1.1. DFA code :

1.1.1.1. Accept strings :

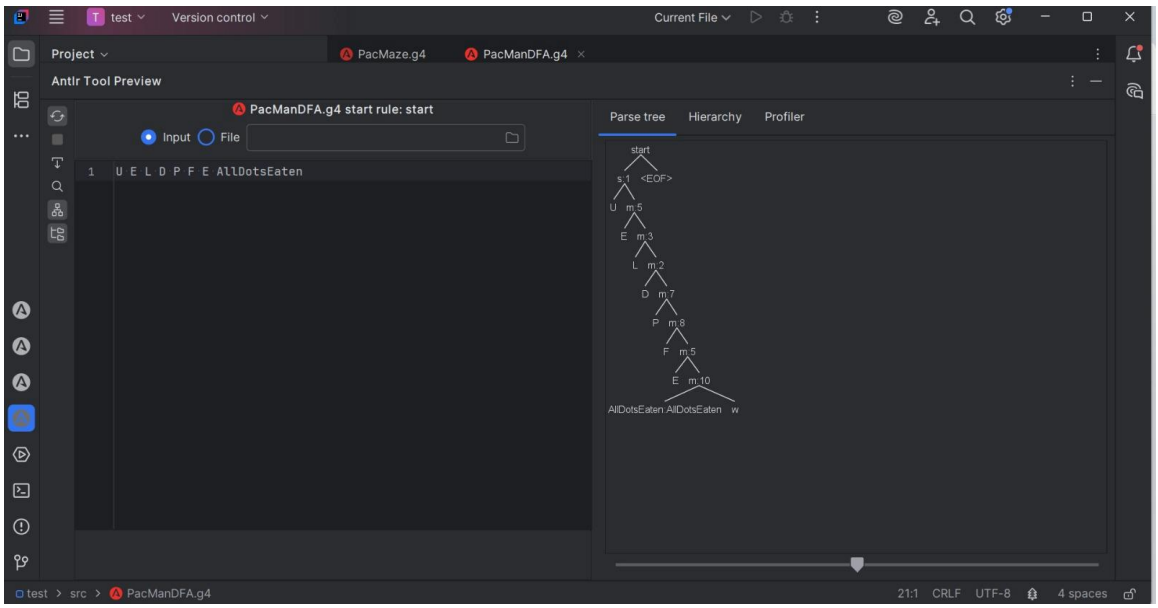


Figure 7 DFA Accept strings

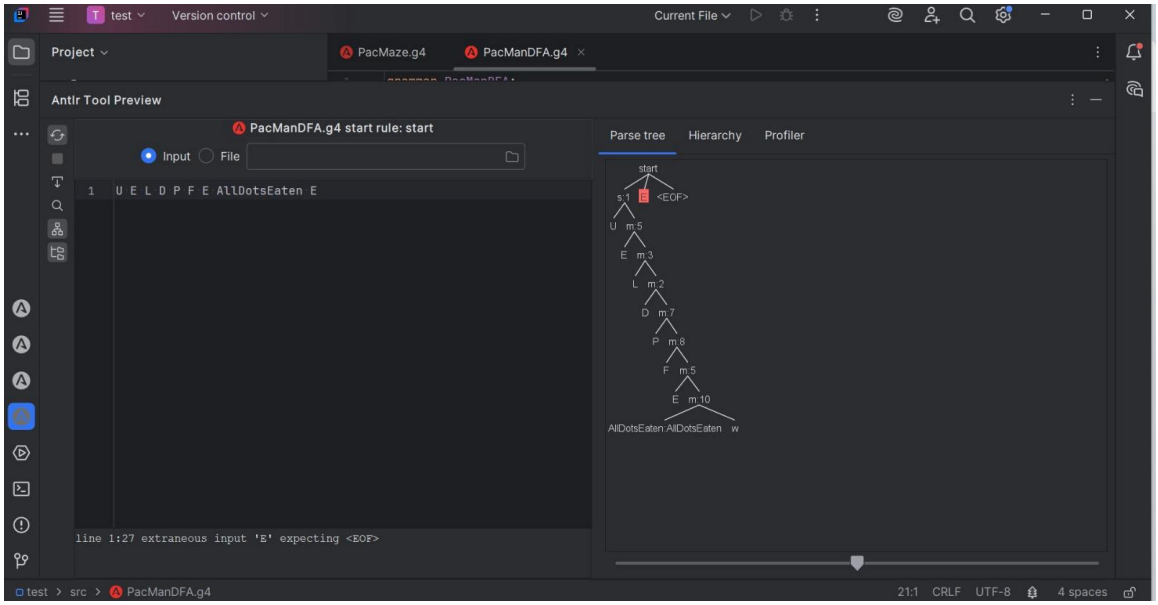


Figure 8 DFA Accept strings (Win state)

1.1.1.2. Rejected strings :

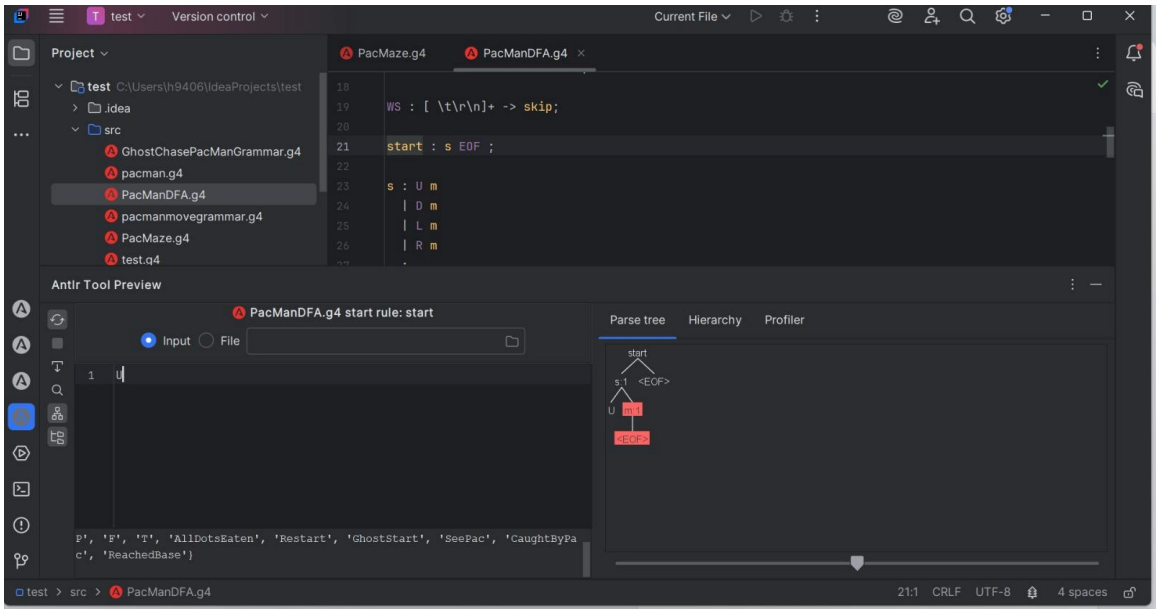


Figure 9 DFA Rejected strings Example

1.1.2. NFA code :

1.1.2.1. Accept strings :

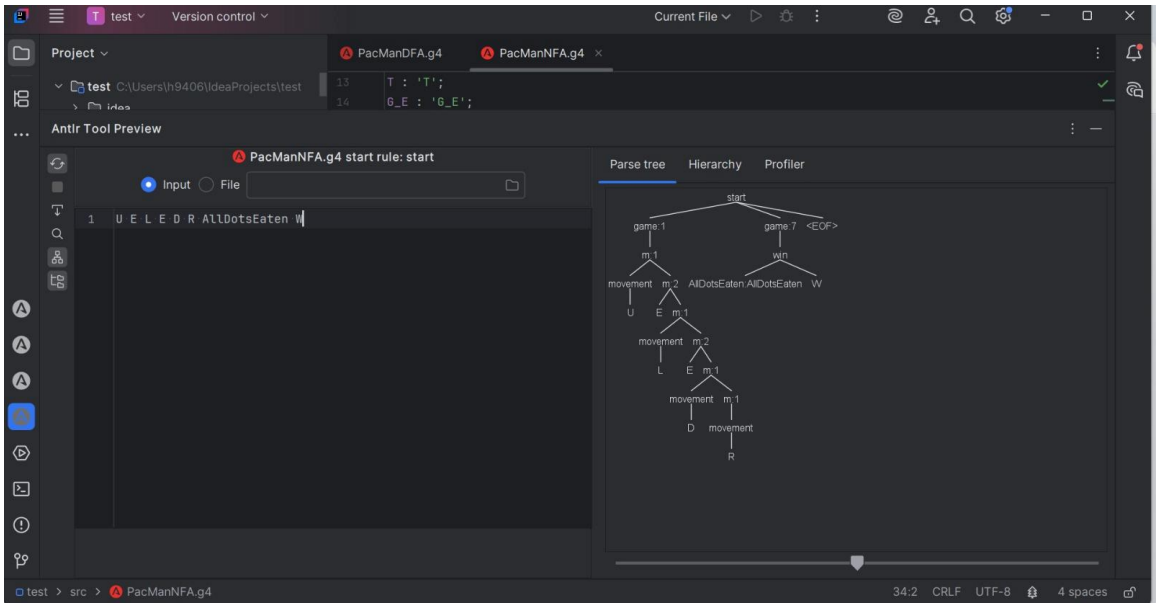


Figure 10 NFA Accept strings Example

1.1.2.2. Rejected strings :

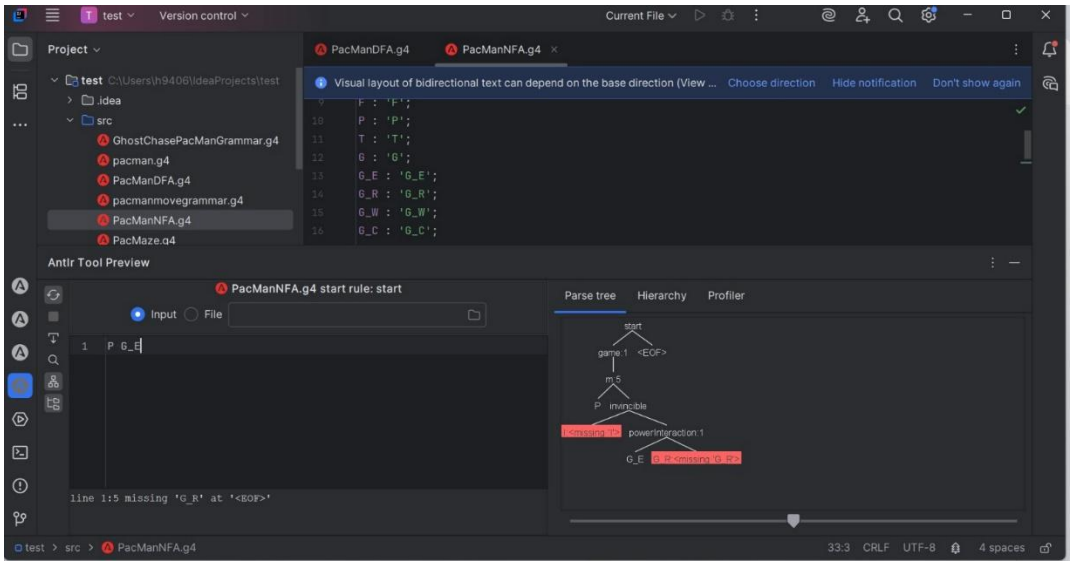


Figure 11 NFA Rejected strings Example

1.2.Website :

The developed website provides an interactive platform for visualizing the parse trees of Pac-Man game logic using formal grammar. Users can input custom grammars and test strings to simulate different gameplay scenarios, such as movement, invincibility, ghost behaviors, win conditions, and game-over sequences. The system analyzes whether the input string is accepted by the grammar and then generates a corresponding parse tree with visual nodes representing each token and rule. This tool is designed to support both educational and technical use cases, allowing users to understand how game state transitions can be modeled and validated using context-free grammars.

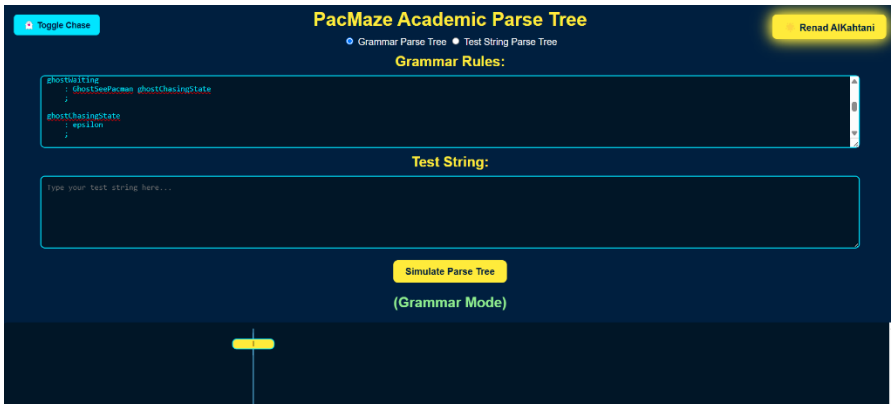


Figure 12 grammar mode

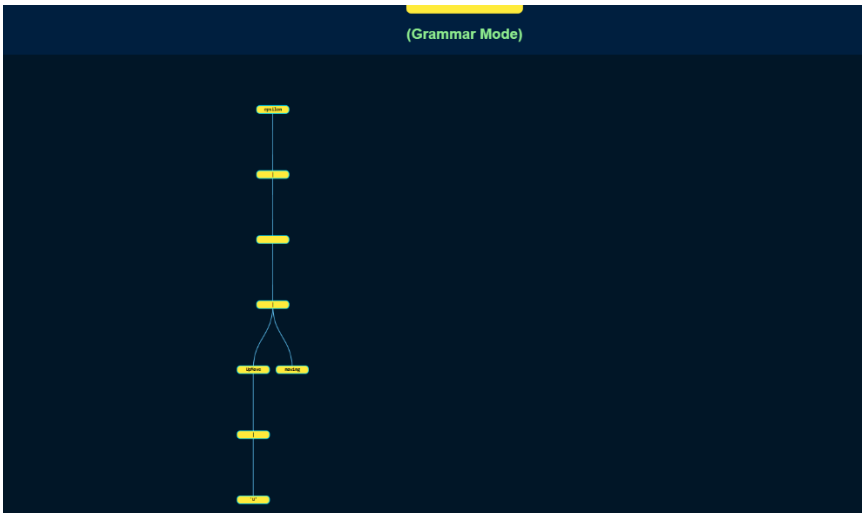


Figure 13 perse tree for grammar

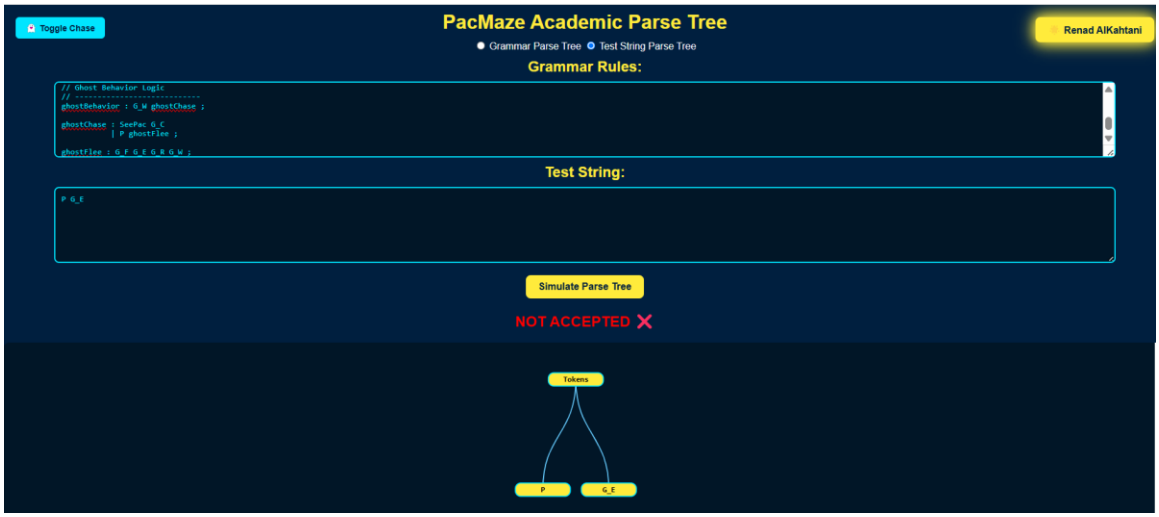


Figure 14 input string perse tree

References :

[1] J. S. L. Tsang, M. A. H. M. Zain, and W. Y. Loh, "A simple tree search method for playing Ms. Pac-Man," Proc. IEEE Int. Conf. on Computational Intelligence and Games, 2010, pp. 215-221.doi: 10.1109/CIG.2010.5593943

[2] T. N. T. T. Le, L. Nguyen, and T. T. Nguyen, "Gameplay analysis using Finite State Machines," *Proc. IEEE Games, Entertainment, and Media Conference*, 2021, pp. 60-66. doi: 10.1109/GEMC.2021.00025